

Watchly

Jalon 3 — Modélisation de la Base de Données

Champ	Valeur
Auteur	MARTIN Gabriel
Formation	CDA — Concepteur Développeur d'Applications
École	IPSSI · [PRS & DIS 26.1] BD3 26.2 · Session Novembre 2025
Date de rendu	31 mars 2026
Version	1.0
Jalon précédent	Jalon 2 — Méthodologie & Conception UX/UI (28/02/2026)
Jalon suivant	Jalon 4 — Réalisation technique (30/04/2026)

CDA — Concepteur Développeur d'Applications

Table des matières

Table des matières	2
I. Introduction	3
1.1 Rappel du projet	3
1.2 Objectifs du Jalon 3	3
1.3 Stratégie de stockage	3
II. Dictionnaire des données	4
2.1 Entité : utilisateur	4
2.2 Entité : film	4
2.3 Entité : genre	5
2.4 Table de liaison : film_genre	5
2.5 Entité : user_collection	5
2.6 Entité : review	6
III. Modèle Conceptuel de Données (MCD)	7
3.1 Diagramme MCD	7
3.2 Description des entités	7
3.3 Description des associations	8
IV. Modèle Logique de Données (MLD)	9
4.1 Diagramme MLD	9
4.2 Schéma relationnel (notation textuelle)	9
4.3 Règles de passage MCD → MLD	10
V. Modèle Physique de Données (MPD)	11
5.1 Scripts SQL de création	11
Table utilisateur	11
Table film	11
Table genre	11
Table film_genre (pivot)	11
Table user_collection	12
Table review	12
VI. Règles de gestion	13
VII. Correspondance Doctrine / Symfony	14
Conclusion	15

I. Introduction

1.1 Rappel du projet

Watchly est une application web de gestion de collection de films développée dans le cadre du projet fil rouge CDA (Concepteur Développeur d'Applications) à l'IPSSI. L'application permet aux utilisateurs de rechercher des films via l'API TMDB, de les ajouter à leur collection personnelle (Watchlist, Films vus, Favoris), de les noter et de rédiger des avis.

L'architecture technique repose sur une API REST développée avec **Symfony** (PHP) et un front-end **React**, le tout conteneurisé avec **Docker**. Les données cinématographiques proviennent de l'**API TMDB** (The Movie Database).

1.2 Objectifs du Jalon 3

Ce troisième jalon a pour objectif de formaliser l'intégralité de la modélisation de la base de données du projet Watchly. Les livrables attendus sont :

- Dictionnaire des données — description exhaustive de chaque attribut de chaque entité
- MCD (Modèle Conceptuel de Données) — vision métier en notation Merise
- MLD (Modèle Logique de Données) — traduction en schéma relationnel
- MPD (Modèle Physique de Données) — implémentation SQL concrète pour MySQL/MariaDB
- Règles de gestion — contraintes métier appliquées aux données

1.3 Stratégie de stockage

Les données cinématographiques proviennent de l'API TMDB. La stratégie retenue est un **cache local enrichi** : les informations essentielles d'un film (titre, synopsis, affiche, date de sortie, durée, note TMDB, genres) sont persistées en base de données dès qu'un utilisateur interagit avec ce film. Les données détaillées (casting complet, bandes-annonces, films similaires) sont récupérées à la volée depuis l'API TMDB lors de la consultation de la fiche film.

Cette approche garantit des temps de chargement rapides pour les listes et collections tout en évitant de dupliquer l'intégralité de la base TMDB localement.

II. Dictionnaire des données

Le dictionnaire des données décrit de manière exhaustive chaque attribut de chaque entité du système. Pour chaque attribut sont précisés le type, les contraintes et une description fonctionnelle.

2.1 Entité : utilisateur

Représente un compte utilisateur de l'application Watchly. Chaque utilisateur peut gérer sa collection de films, attribuer des notes et rédiger des avis.

Attribut	Type	Null	Contrainte	Description
id	INT	Non	PK, AI	Identifiant unique auto-incrémenté
email	VARCHAR(180)	Non	UNIQUE	Adresse email de connexion
password	VARCHAR(255)	Non	—	Mot de passe hashé (bcrypt via Symfony)
username	VARCHAR(50)	Non	UNIQUE	Pseudo affiché publiquement
roles	JSON	Non	Défaut []	Rôles Symfony (ROLE_USER, ROLE_ADMIN)
created_at	DATETIME	Non	—	Date de création du compte
updated_at	DATETIME	Non	—	Date de dernière modification

2.2 Entité : film

Cache local des films issus de l'API TMDB. Un film est persisté en base dès qu'un utilisateur interagit avec lui (ajout en collection, notation ou rédaction d'avis).

Attribut	Type	Null	Contrainte	Description
id	INT	Non	PK, AI	Identifiant unique auto-incrémenté
tmdb_id	INT	Non	UNIQUE	Identifiant du film sur TMDB
title	VARCHAR(255)	Non	—	Titre du film (en français)
original_title	VARCHAR(255)	Oui	—	Titre original (VO)
overview	TEXT	Oui	—	Synopsis du film
poster_path	VARCHAR(255)	Oui	—	Chemin de l'affiche (TMDB CDN)
backdrop_path	VARCHAR(255)	Oui	—	Chemin de l'image de fond (TMDB CDN)
release_date	DATE	Oui	—	Date de sortie du film
runtime	INT	Oui	—	Durée en minutes
vote_average	DECIMAL(3,1)	Oui	—	Note moyenne TMDB (0.0 à 10.0)

Attribut	Type	Null	Contrainte	Description
created_at	DATETIME	Non	—	Date d'ajout en base locale

2.3 Entité : genre

Référentiel des genres cinématographiques, synchronisé depuis l'API TMDB. Permet le filtrage et le classement des films par catégorie.

Attribut	Type	Null	Contrainte	Description
id	INT	Non	PK, AI	Identifiant unique auto-incrémenté
tmdb_id	INT	Non	UNIQUE	Identifiant du genre sur TMDB
name	VARCHAR(100)	Non	—	Libellé du genre (ex : Action, Comédie)

2.4 Table de liaison : film_genre

Table pivot matérialisant la relation Many-to-Many entre les films et les genres. Un film peut appartenir à plusieurs genres, et un genre regroupe plusieurs films.

Attribut	Type	Null	Contrainte	Description
film_id	INT	Non	PK, FK	Référence vers film.id
genre_id	INT	Non	PK, FK	Référence vers genre.id

2.5 Entité : user_collection

Représente l'entrée d'un film dans la collection d'un utilisateur. Chaque entrée possède un statut exclusif (Watchlist, Vu ou Favori) et une note personnelle optionnelle.

Attribut	Type	Null	Contrainte	Description
id	INT	Non	PK, AI	Identifiant unique auto-incrémenté
user_id	INT	Non	FK	Référence vers utilisateur.id
film_id	INT	Non	FK	Référence vers film.id
status	ENUM	Non	—	Statut : WATCHLIST, WATCHED ou FAVORITE
rating	SMALLINT	Oui	CHECK 1-5	Note personnelle de 1 à 5 étoiles
added_at	DATETIME	Non	—	Date d'ajout à la collection
watched_at	DATETIME	Oui	—	Date de visionnage (null si WATCHLIST)

2.6 Entité : review

Avis textuel rédigé par un utilisateur sur un film. Un utilisateur ne peut rédiger qu'un seul avis par film.

Attribut	Type	Null	Contrainte	Description
id	INT	Non	PK, AI	Identifiant unique auto-incrémenté
user_id	INT	Non	FK	Référence vers utilisateur.id
film_id	INT	Non	FK	Référence vers film.id
content	TEXT	Non	—	Contenu de l'avis rédigé par l'utilisateur
created_at	DATETIME	Non	—	Date de création de l'avis
updated_at	DATETIME	Non	—	Date de dernière modification

III. Modèle Conceptuel de Données (MCD)

Le MCD (notation Merise) représente la vision métier du système d'information. Il identifie les **entités** (rectangles), les **associations** (losanges) et les **cardinalités** qui régissent les relations entre elles.

3.1 Diagramme MCD

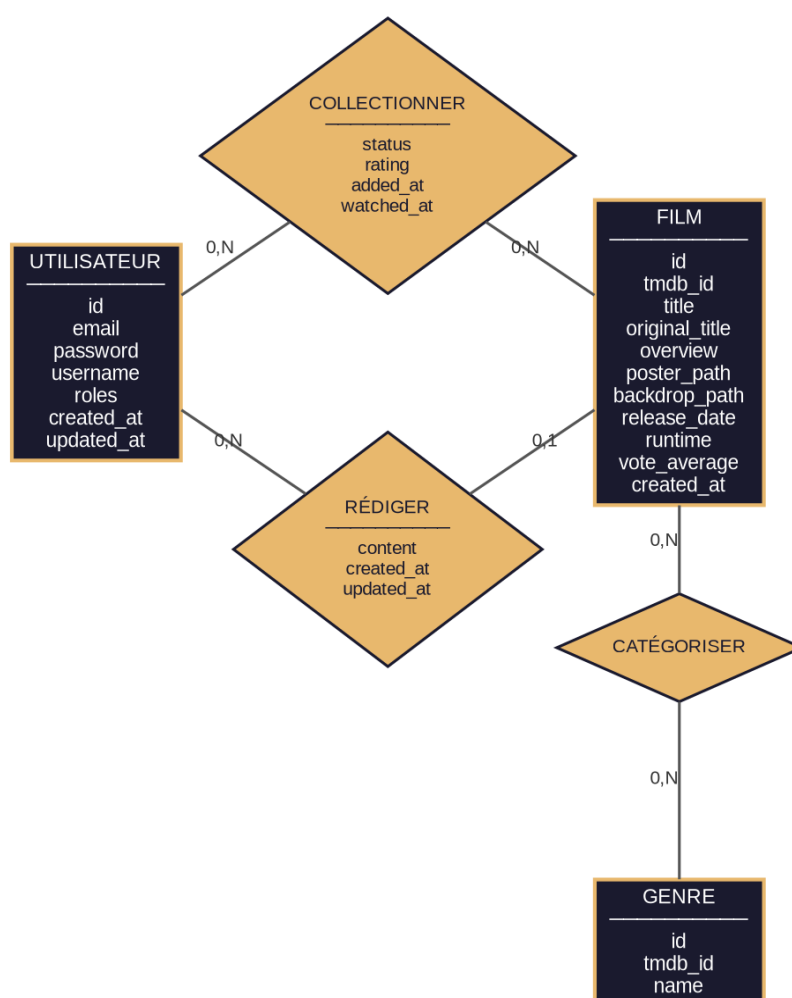


Figure 1 — MCD Merise de Watchly

3.2 Description des entités

Entité	Description	Attributs clés
UTILISATEUR	Compte utilisateur de l'application	id, email, username, password, roles

Entité	Description	Attributs clés
FILM	Cache local d'un film TMDB	id, tmdb_id, title, overview, poster_path
GENRE	Catégorie cinématographique TMDB	id, tmdb_id, name

3.3 Description des associations

Association	Entités reliées	Cardinalités	Attributs propres
COLLECTIONNER	UTILISATEUR ↔ FILM	0,N — 0,N	status, rating, added_at, watched_at
RÉDIGER	UTILISATEUR ↔ FILM	0,N — 0,1	content, created_at, updated_at
CATÉGORISER	FILM ↔ GENRE	0,N — 0,N	Aucun (association simple)

La cardinalité 0,1 côté FILM dans l'association RÉDIGER signifie qu'un utilisateur ne peut rédiger qu'un seul avis par film (contrainte d'unicité sur le couple user_id / film_id).

IV. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en un schéma relationnel exploitable par un SGBD. Les entités deviennent des **tables**, les associations porteuses d'attributs deviennent des **tables associatives**, et les clés étrangères matérialisent les relations.

4.1 Diagramme MLD

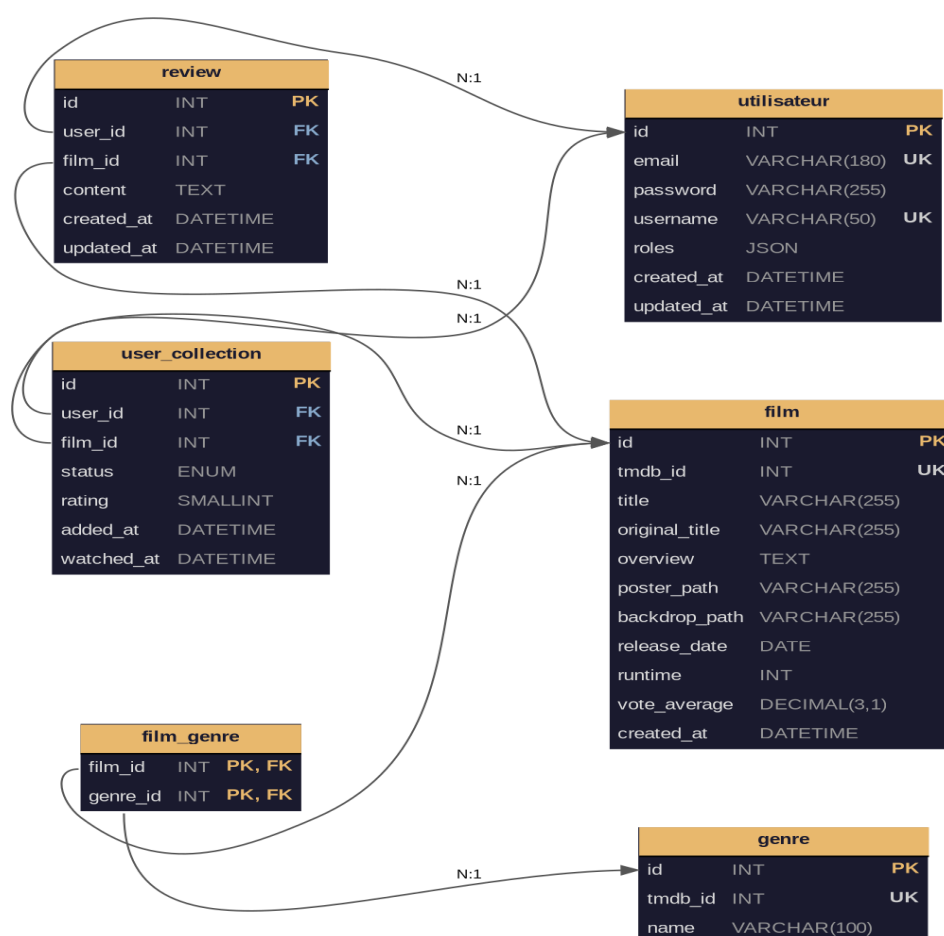


Figure 2 — Schéma relationnel (MLD) de Watchly

4.2 Schéma relationnel (notation textuelle)

Le schéma relationnel est exprimé ci-dessous en notation formelle. Les clés primaires sont soulignées, les clés étrangères préfixées par le symbole #.

```

utilisateur (id, email, password, username, roles, created_at, updated_at)
film (id, tmdb_id, title, original_title, overview, poster_path, backdrop_path,
release_date, runtime, vote_average, created_at)
genre (id, tmdb_id, name)
film_genre (#film_id, #genre_id)
  
```

user_collection (id, #user_id, #film_id, status, rating, added_at, watched_at)
review (id, #user_id, #film_id, content, created_at, updated_at)

4.3 Règles de passage MCD → MLD

Élément MCD	Transformation MLD	Résultat
Entité UTILISATEUR	Entité → Table	Table utilisateur avec PK id
Entité FILM	Entité → Table	Table film avec PK id
Entité GENRE	Entité → Table	Table genre avec PK id
Assoc. COLLECTIONNER (0,N—0,N)	Assoc. N:N avec attributs → Table	Table user_collection avec FK user_id, film_id
Assoc. RÉDIGER (0,N—0,1)	Assoc. avec attributs → Table	Table review avec FK user_id, film_id + UNIQUE
Assoc. CATÉGORISER (0,N—0,N)	Assoc. N:N simple → Table pivot	Table film_genre avec PK composite

V. Modèle Physique de Données (MPD)

Le MPD traduit le MLD en instructions SQL exécutables. Le SGBD cible est **MySQL 8.0 / MariaDB 10.6**, compatible avec l'ORM **Doctrine** utilisé par Symfony. Le moteur de stockage est **InnoDB** pour le support des transactions et des clés étrangères.

5.1 Scripts SQL de création

En pratique, ces tables seront générées automatiquement par les migrations Doctrine à partir des entités PHP. Le SQL ci-dessous sert de référence documentaire.

Table utilisateur

```
CREATE TABLE utilisateur (
  id INT AUTO_INCREMENT NOT NULL,
  email VARCHAR(180) NOT NULL,
  password VARCHAR(255) NOT NULL,
  username VARCHAR(50) NOT NULL,
  roles JSON NOT NULL DEFAULT '[]',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE INDEX uq_user_email (email),
  UNIQUE INDEX uq_user_username (username),
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Table film

```
CREATE TABLE film (
  id INT AUTO_INCREMENT NOT NULL,
  tmdb_id INT NOT NULL,
  title VARCHAR(255) NOT NULL,
  original_title VARCHAR(255) DEFAULT NULL,
  overview TEXT DEFAULT NULL,
  poster_path VARCHAR(255) DEFAULT NULL,
  backdrop_path VARCHAR(255) DEFAULT NULL,
  release_date DATE DEFAULT NULL,
  runtime INT DEFAULT NULL,
  vote_average DECIMAL(3,1) DEFAULT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE INDEX uq_film_tmdb (tmdb_id),
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Table genre

```
CREATE TABLE genre (
  id INT AUTO_INCREMENT NOT NULL,
  tmdb_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  UNIQUE INDEX uq_genre_tmdb (tmdb_id),
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Table film_genre (pivot)

```
CREATE TABLE film_genre (
  film_id INT NOT NULL,
  genre_id INT NOT NULL,
  PRIMARY KEY (film_id, genre_id),
```

```
INDEX idx_fg_film (film_id),
INDEX idx_fg_genre (genre_id),
CONSTRAINT fk_fg_film FOREIGN KEY (film_id)
    REFERENCES film (id) ON DELETE CASCADE,
CONSTRAINT fk_fg_genre FOREIGN KEY (genre_id)
    REFERENCES genre (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Table user_collection

```
CREATE TABLE user_collection (
    id INT AUTO_INCREMENT NOT NULL,
    user_id INT NOT NULL,
    film_id INT NOT NULL,
    status ENUM('WATCHLIST', 'WATCHED', 'FAVORITE') NOT NULL,
    rating SMALLINT DEFAULT NULL,
    added_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    watched_at DATETIME DEFAULT NULL,
    UNIQUE INDEX uq_uc_user_film (user_id, film_id),
    INDEX idx_uc_user (user_id),
    INDEX idx_uc_film (film_id),
    INDEX idx_uc_status (status),
    CONSTRAINT fk_uc_user FOREIGN KEY (user_id)
        REFERENCES utilisateur (id) ON DELETE CASCADE,
    CONSTRAINT fk_uc_film FOREIGN KEY (film_id)
        REFERENCES film (id) ON DELETE CASCADE,
    CONSTRAINT chk_uc_rating CHECK (rating BETWEEN 1 AND 5),
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Table review

```
CREATE TABLE review (
    id INT AUTO_INCREMENT NOT NULL,
    user_id INT NOT NULL,
    film_id INT NOT NULL,
    content TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE INDEX uq_review_user_film (user_id, film_id),
    INDEX idx_review_user (user_id),
    INDEX idx_review_film (film_id),
    CONSTRAINT fk_review_user FOREIGN KEY (user_id)
        REFERENCES utilisateur (id) ON DELETE CASCADE,
    CONSTRAINT fk_review_film FOREIGN KEY (film_id)
        REFERENCES film (id) ON DELETE CASCADE,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

VI. Règles de gestion

Les règles de gestion définissent les contraintes métier appliquées aux données, au-delà des simples contraintes d'intégrité relationnelle.

Règle	Intitulé	Description
RG-01	Unicité email	Un utilisateur est identifié de manière unique par son adresse email. Deux comptes ne peuvent pas partager la même adresse.
RG-02	Unicité username	Le pseudo d'un utilisateur doit être unique dans l'application.
RG-03	Statuts exclusifs	Un film dans la collection d'un utilisateur possède un statut unique parmi WATCHLIST, WATCHED ou FAVORITE. Un film en WATCHLIST ne peut pas être WATCHED ou FAVORITE simultanément.
RG-04	Favori implique vu	Le statut FAVORITE implique que le film a été vu. C'est un statut « Vu + Coup de cœur ».
RG-05	Note conditionnelle	Un utilisateur ne peut noter un film (1 à 5 étoiles) que si le film a le statut WATCHED ou FAVORITE. Un film en WATCHLIST ne peut pas avoir de note.
RG-06	Un avis par film	Un utilisateur ne peut rédiger qu'un seul avis par film (contrainte UNIQUE sur user_id + film_id dans review).
RG-07	Film unique TMDB	Un film TMDB n'est persisté qu'une seule fois en base (UNIQUE sur tmdb_id). Plusieurs utilisateurs partagent la même référence film.
RG-08	Suppression en cascade	La suppression d'un utilisateur entraîne la suppression de sa collection et de ses avis. La suppression d'un film entraîne la suppression des entrées de collection et des avis associés.
RG-09	Genres TMDB	Les genres sont importés et synchronisés depuis l'API TMDB. Ils ne sont pas modifiables par les utilisateurs.
RG-10	Date de visionnage	Le champ watched_at est renseigné automatiquement lors du passage au statut WATCHED ou FAVORITE, et reste null pour le statut WATCHLIST.

VII. Correspondance Doctrine / Symfony

Le MPD sera implémenté via l'ORM **Doctrine** intégré à Symfony. Le tableau suivant établit la correspondance entre les tables SQL et les entités PHP qui seront créées.

Table SQL	Entité Doctrine	Relations Doctrine	Annotations clés
utilisateur	User	OneToMany → UserCollection, Review	UserInterface, PasswordAuthenticatedUserInterface
film	Film	OneToMany → UserCollection, Review ManyToMany → Genre	Index sur tmdb_id
genre	Genre	ManyToMany → Film	Importé depuis TMDB
film_genre	(table gérée par Doctrine)	ManyToMany automatique	JoinTable entre Film et Genre
user_collection	UserCollection	ManyToOne → User, Film	UniqueConstraint sur (user, film)
review	Review	ManyToOne → User, Film	UniqueConstraint sur (user, film)

Les entités Doctrine seront générées avec le `maker bundle` de Symfony (`php bin/console make:entity`) puis ajustées manuellement pour les contraintes avancées (ENUM, CHECK, indexes composés).

Conclusion

Ce livrable de Jalon 3 formalise l'intégralité de la modélisation de la base de données du projet **Watchly**. Le dictionnaire des données décrit chaque attribut de manière exhaustive, le MCD en notation Merise capture la vision métier, le MLD traduit cette vision en schéma relationnel, et le MPD fournit les scripts SQL prêts à l'emploi pour MySQL/MariaDB.

La base de données a été conçue pour s'articuler avec l'API TMDB selon une stratégie de cache local enrichi, garantissant à la fois la performance (accès rapide aux collections) et la fraîcheur des données (appels TMDB pour les fiches détaillées). Les règles de gestion assurent la cohérence métier, notamment l'exclusivité des statuts et l'unicité des avis par utilisateur.

Ce modèle servira de fondation pour le développement des entités Doctrine et des migrations Symfony lors du **Jalon 4 — Réalisation technique** (avril 2026).

Document rédigé dans le cadre du Projet Fil Rouge CDA · IPSSI · Mars 2026